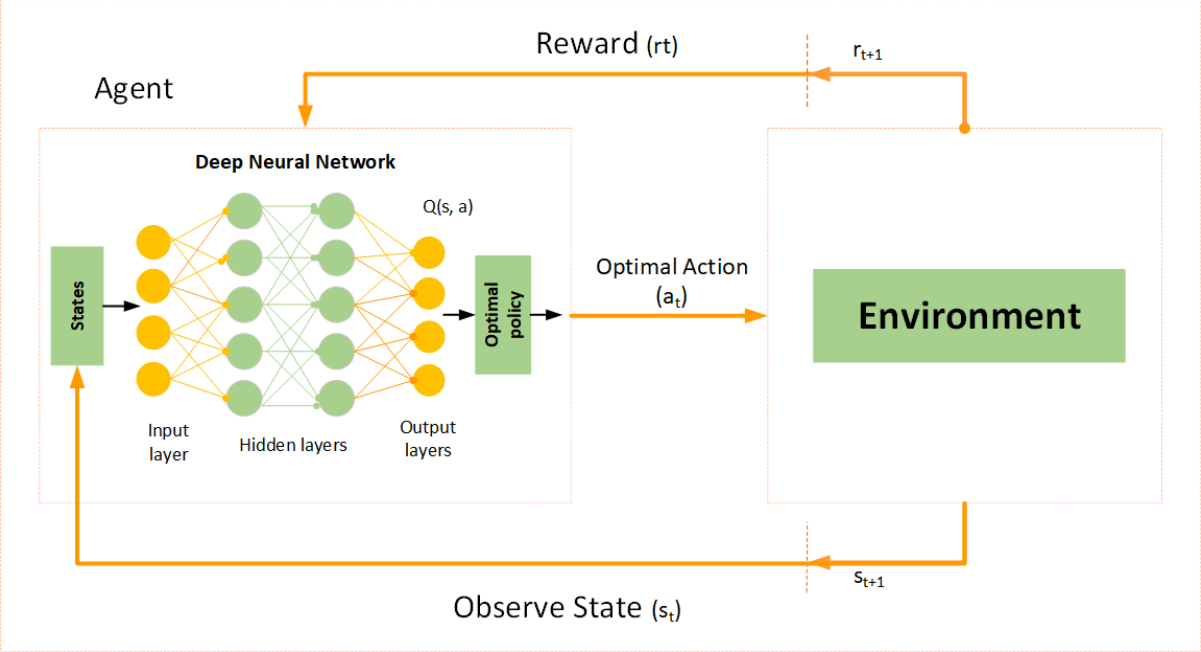
Université Chouaïb Doukkali
Faculté des Sciences El Jadida

Projet : Deep Reinforcement Learning applique au jeu de Dama à l'aide d'un modèle DQN

Modèle 117 : Intelligence Artificielle applique à la physique



Préparé par G4:

- **Abdelali Chouaib**
- **Idrissi Dafali Hakim**
- **Oumma Ayoub**
- **Tamim Ikram**
- **Zineb Charaf**

✓ **Supervisé par : Pr.Otmane Houdaif**

Première année du Master d'Excellence (2E2I)

Résumé :

Ce projet présente la conception, l'entraînement et le déploiement d'un agent d'intelligence artificielle capable de jouer de manière autonome à la Dama (variante marocaine du jeu de dames). Pour relever le défi posé par la complexité de l'espace d'états de ce jeu, nous avons implémenté un réseau de neurones convolutif (CNN) couplé à l'algorithme d'apprentissage par renforcement **Double DQN (Double Deep Q-Network)**. Afin de stabiliser et d'accélérer la convergence du modèle, nous avons élaboré un pipeline d'entraînement novateur structuré en trois phases successives : un pré-entraînement par apprentissage supervisé (Imitation d'un algorithme Minimax), une phase d'exploration autonome (*Self-Play*), et un affinement par apprentissage progressif (*Curriculum Learning* adaptatif). Entraîné en 197 minutes grâce à l'accélération GPU sur Google Colab (4000 épisodes au total), le modèle démontre une excellente assimilation des règles et des tactiques de base, atteignant un taux de victoire supérieur à 70% face à des adversaires aléatoires, tout en opposant une résistance stratégique face à un algorithme Minimax de niveau supérieur. Enfin, le réseau de neurones a été exporté au format léger TensorFlow.js, prouvant la faisabilité d'un déploiement *AI* fluide et exécutable directement dans un navigateur web sans serveur de calcul distant.

Le code source utilisé au cours de ce Projet est disponible en ligne via le lien Colab suivant :

Table de Contenus

Résumé : 1
I. Introduction & Base Théorique 3
1. Introduction 3
1.1 Problématique : Pourquoi le jeu de Dama ? 4
2. Base Théorique 5
2.1 Reinforcement Learning : 5
2.2 L'algorithme d'apprentissage : 6
II. Implémentation et Protocole d'Entraînement 8
1. Environnement de développement et infrastructure : 8
1.1 Dépendances et Bibliothèques : 9
1.2 Global Hyperparameters: 10
2. L'environnement Checkers (DamaEnv) : 11
2.1 Représentation du Plateau (Board Representation) : 11
2.2 Espace des Actions (Action Space) *:* 11
2.3 Génération des Mouvements et Règles Spécifiques : 12
2.4 Masquage des Actions (Action Masking): 12
3. Architecture du Réseau de Neurones : 13
3.1 Planification du Taux d'Apprentissage (Learning Rate Schedule): 14
3.2 Mémoire d'Expérience (Replay Buffer) : 15
3.3 Implémentation de l'Algorithme Minimax : 16
4. Entraînement : 17
Phase A — Apprentissage par Imitation (*Imitation Learning*) : 17
Phase B — Apprentissage Autonome (*Self-Play DQN*) : 20
Phase C — Apprentissage par Curriculum Adaptatif 20
5. Évaluation et Visualisation des Performances et Déploiement Web: 21
Déploiement Web : Exportation vers TensorFlow.js 22
Conclusion : 23
Références bibliographiques 24
Annexe 1 : Justification théorique et paramétrage algorithmique du modèle 25
Annexe 2 : algorithme Mini-MAX 27

I. Introduction & Base Théorique

1. Introduction

Ce projet s'inscrit dans le domaine de l'Intelligence Artificielle et explore plus particulièrement les concepts de l'apprentissage par renforcement (*Reinforcement Learning*). Notre objectif principal est de concevoir, d'entraîner et de déployer un agent autonome capable de maîtriser le jeu de Dama (variante marocaine du jeu de dames), en s'appuyant sur l'architecture algorithmique **Double DQN (Double Deep Q-Network)**.

Pour surmonter la complexité du jeu et optimiser la convergence de notre intelligence artificielle, nous avons élaboré une stratégie d'entraînement rigoureuse divisée en trois phases progressives :

- **Phase 1 : Apprentissage par Imitation (Supervisé) :** Pour initialiser le "cerveau" de l'agent et lui donner des bases solides, la première phase consiste à lui faire observer et imiter un algorithme classique (Minimax configuré à une profondeur de 2). L'agent apprend ainsi les règles fondamentales et les tactiques de base avant d'être livré à lui-même.
- **Phase 2 : Apprentissage Autonome (*Self-Play*) :** Une fois les bases acquises, l'agent bascule en pur apprentissage par renforcement. Il s'affronte lui-même lors de milliers de parties. Cette phase d'exploration libre lui donne l'opportunité de découvrir des stratégies inédites et de dépasser les limites de l'algorithme Minimax qu'il imitait au départ.
- **Phase 3 : Apprentissage par Curriculum (*Curriculum Learning*) :** Pour consolider sa robustesse, l'agent subit un entraînement final "à la carte". Il affronte alternativement ses propres versions antérieures (*Self-Play*), puis un algorithme Minimax de niveau faible (profondeur 1), et enfin un Minimax de niveau supérieur (profondeur 2). Cette montée en difficulté graduelle force le modèle à s'adapter en permanence à divers styles de jeu.

Enfin, pour valider notre démarche scientifique, le projet intègre la génération d'un **tableau de bord (*dashboard*) complet**. Celui-ci permet de visualiser et d'évaluer l'évolution des paramètres et des performances de l'agent au fil de son apprentissage.

L'aboutissement technique de ce projet réside dans l'exportation du modèle neuronal final (via TensorFlow.js) afin de l'intégrer directement dans une **application web front-end**, permettant à n'importe quel utilisateur humain d'affronter notre IA de manière fluide depuis son navigateur.

1.1 Problématique : Pourquoi le jeu de Dama ?

Le choix du jeu de Dama comme environnement d'apprentissage repose sur plusieurs raisons importantes, détaillées ci-après.

➢ *Un espace d'états extrêmement vaste :*

Le jeu de Dama possède un espace d'états très large, correspondant au nombre total de configurations possibles du plateau de jeu. Chaque configuration dépend de la position des pions, du nombre de pièces restantes et de leur statut (pion simple ou dame).

Dans les jeux de stratégie comme Dama, ce nombre de configurations peut atteindre des valeurs très élevées, pouvant aller jusqu'à $\mathbf{10^{18}}$ **combinaisons possibles**. Cette complexité rend impossible l'exploration exhaustive de toutes les situations du jeu à l'aide de méthodes traditionnelles.

Par conséquent, une intelligence artificielle doit être capable de généraliser son apprentissage afin de prendre des décisions efficaces même dans des situations jamais rencontrées auparavant.

➢ Difficulté de programmer toutes les stratégies :

Une autre difficulté importante réside dans le fait qu'il est extrêmement difficile de coder manuellement toutes les stratégies de jeu. Les approches basées uniquement sur des règles prédéfinies nécessitent l'identification et la programmation de nombreux scénarios possibles.

Cependant, dans un jeu complexe comme Dama, plusieurs facteurs rendent cette approche impraticable :

- Les situations de jeu sont très variées et multiples ;
- Les décisions optimales dépendent fortement du contexte de la partie ;
- Les stratégies évoluent en fonction de l'état courant du jeu.

Ainsi, la programmation manuelle de toutes les règles stratégiques devient rapidement impraticable et ne permet pas d'obtenir une intelligence artificielle suffisamment flexible et adaptable.

2. Base Théorique

2.1 Reinforcement Learning :

L'apprentissage par renforcement (ou *Reinforcement Learning - RL*) constitue le troisième grand paradigme de l'intelligence artificielle, distinct de l'apprentissage supervisé et de l'apprentissage non supervisé.

Apprentissage supervisé vs Apprentissage par renforcement :

Dans l'**apprentissage supervisé**, l'algorithme apprend à partir d'un ensemble de données "étiquetées" par un expert humain (un professeur). On lui montre par exemple des milliers de coups d'échecs en lui indiquant exactement quel est le "bon" coup à jouer.

À l'inverse, l'**apprentissage par renforcement** ne dispose d'aucune donnée pré-étiquetée. L'agent (l'IA) apprend de manière totalement autonome par **essais et erreurs**. Il interagit directement avec son environnement et découvre par lui-même quelles stratégies fonctionnent ou échouent en fonction des conséquences de ses actes.
Le cycle d'interaction (État, Action, Récompense)

Le fonctionnement de l'apprentissage par renforcement repose sur un cycle mathématique continu (souvent modélisé par un Processus de Décision Markovien) impliquant un **Agent** (notre IA) et un **Environnement** (notre jeu de Dama). Ce cycle se définit par trois concepts fondamentaux :

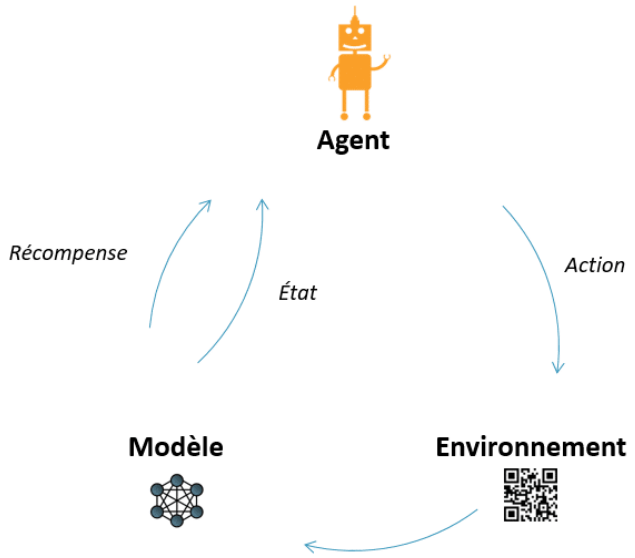


Figure 1 Schéma Agent/Environnement

- **L'État (s pour *State*) :** C'est l'observation de l'environnement à un instant donné **t**. C'est la "photographie" de la situation à laquelle l'agent est confronté avant de prendre une décision.
  - *Dans notre projet (Dama) :* **L'état s** correspond à la configuration exacte du damier (la position des pions rouges, des pions blancs, des dames, et des cases vides) sous forme d'une matrice 8x8.
- **L'Action (a pour *Action*) :** C'est la décision prise par l'agent en réponse à **l'état s**. L'agent choisit cette action parmi un ensemble de coups légaux possibles dans le but de maximiser ses chances de succès.
  - *Dans notre projet (Dama) :* L'action a correspond au déplacement d'un pion spécifique d'une case de départ vers une case d'arrivée (incluant les éventuelles captures).

- **La Récompense (r pour *Reward*) :** C'est le signal de retour, sous forme de valeur numérique scalaire, envoyé par l'environnement immédiatement après que l'agent ait exécuté son **action a**. Ce signal indique à l'agent si l'action qu'il vient de faire était bonne (récompense positive) ou mauvaise (pénalité). Le but ultime de l'agent est de maximiser la somme totale des récompenses accumulées sur le long terme.
  - *Dans notre projet (Dama) :* La récompense est finement réglée pour dicter le comportement de l'IA : elle gagne par exemple r = +0.3 si elle capture un pion adverse, r = +0.5 si elle crée une Dame, r = +1.0 si elle gagne la partie, et subit une très légère pénalité r = -0.002 à chaque tour pour l'encourager à jouer de manière agressive et rapide.

2.2 L'algorithme d'apprentissage :

Historiquement, l'apprentissage par renforcement s'appuyait sur le **Q-Learning classique**, qui utilise une **Matrice Q (Q-Table)**. Cette matrice agit comme un immense tableau de correspondances : chaque ligne représente une configuration exacte du plateau (un état) et chaque colonne représente un coup possible (une action). À l'intersection se trouve le "score" estimé de ce coup.

Pour calculer et mettre à jour ces scores au fil des parties, l'algorithme s'appuie sur le cœur mathématique du Q-Learning : **l'équation de Bellman**. Dans sa forme simplifiée, l'objectif d'apprentissage s'exprime ainsi :

$$Q(s,a) = R + \gamma \max Q(s',a') \,.$$

Cette équation stipule que la valeur d'un **coup a** dans un **état s** (le fameux **Q(s,a)**) est égale à la somme de deux éléments :

- **R (Récompense immédiate) :** Les points gagnés instantanément par ce coup (ex: +0.3 pour une capture).

- **γ max Q (s', a')(Récompense future estimée) :** La valeur du meilleur coup possible au tour suivant (dans le nouvel état s'), pondérée par un facteur d'escompte γ (gamma, souvent fixé à 0.95). Ce paramètre γ est crucial : il force l'agent à avoir une vision stratégique à long terme (anticiper une victoire) plutôt que de se jeter aveuglément sur le premier gain immédiat.

Cependant, cette approche classique par tableau se heurte rapidement à la "malédiction de la dimensionnalité". Le jeu de Dama possède un nombre astronomique de combinaisons possibles (des milliards d'états). Une Matrice Q nécessiterait une mémoire informatique irréaliste pour tout stocker et un temps d'entraînement infini pour explorer chaque case.

Pour surmonter cette limitation, nous avons recours au **Deep Q-Learning (DQN)**, qui remplace cette gigantesque matrice par un **Réseau de Neurones Artificiels**. Au lieu de chercher la réponse exacte dans un tableau, l'agent "observe" le plateau en entrée (*input* du réseau), et le réseau calcule mathématiquement, à travers ses différentes couches, une estimation des scores pour toutes les actions possibles en sortie (*output*). Durant son entraînement, le réseau ajuste ses connexions internes (ses poids) pour que ses prédictions se rapprochent le plus possible du résultat idéal dicté par l'équation de Bellman.

Ce passage de la matrice au réseau de neurones offre deux avantages majeurs :

- **La capacité de généralisation :** C'est l'atout principal. Une Matrice Q est "aveugle" face à une situation qu'elle n'a jamais rencontrée. À l'inverse, un réseau de neurones est capable de reconnaître des "motifs" (des alignements de pions, des pièges) et de déduire un excellent coup face à une configuration de plateau totalement inédite, en se basant sur son expérience de situations similaires.
- **L'efficacité mémoire :** Au lieu de stocker des milliards d'états dans la RAM, nous ne stockons que l'architecture et les "poids" du réseau de neurones, aboutissant à un fichier très léger (quelques mégaoctets) parfaitement adapté à une exécution fluide sur un navigateur web

2.2.1 Le choix de l'algorithme : Pourquoi le Double DQN ?

Pour concevoir le « cerveau » de notre agent, notre choix s'est porté sur l'algorithme **Double DQN (Double Deep Q-Network)**. Bien qu'il existe de nombreuses architectures modernes en apprentissage par renforcement, le Double DQN offre le meilleur compromis entre stabilité mathématique, rapidité d'entraînement et adéquation avec un jeu de plateau.

Voici pourquoi nous l'avons privilégié face aux autres standards du domaine :

- ✓ **Face au DQN (Standard) : La correction du biais de surestimation** L'algorithme DQN classique utilise le même réseau de neurones pour choisir la meilleure action et pour en évaluer sa valeur. Mathématiquement, cela crée un "biais de maximisation" : l'algorithme a tendance à être trop optimiste et à surestimer de manière chronique les récompenses futures, ce qui finit par déstabiliser totalement l'apprentissage. Le Double DQN corrige élégamment cette faille en séparant les rôles : le réseau principal *choisit* l'action, et un second réseau (le *target network*) l'*évalue*. L'apprentissage devient ainsi beaucoup plus stable, réaliste et évite l'effondrement des performances.

- ✓ **Face au Dueling DQN : L'équilibre entre performance et légèreté** Le Dueling DQN est une évolution du DQN qui sépare les flux d'informations à l'intérieur même du réseau : d'un côté l'évaluation globale de l'état du plateau (*Value*), de l'autre l'avantage spécifique d'un coup particulier (*Advantage*). Bien que très performante, cette architecture ajoute une couche de complexité et de lourdeur de calcul. Pour un espace d'état déterministe tel qu'un damier 8x8 de Dama, l'architecture du Double DQN s'est avérée amplement suffisante. Elle permet des itérations d'entraînement beaucoup plus rapides sans sacrifier la qualité stratégique, tout en générant un modèle final plus léger pour le navigateur web.

- ✓ **Face au PPO (Proximal Policy Optimization) : La pertinence face aux espaces discrets** Le PPO est aujourd'hui l'un des algorithmes les plus avancés et populaires (utilisé notamment pour entraîner ChatGPT ou des IA robotiques). Toutefois, il appartient à la famille des gradients de politique (*Policy Gradients*). Il est très lourd en mémoire, très sensible à ses hyperparamètres, et brille particulièrement dans les environnements continus et imprévisibles. Pour un jeu de société au tour par tour, strictement déterministe et possédant un espace d'actions "discret" (déplacer un pion d'une case fixe A à une case B), une méthode basée sur l'estimation de la valeur mathématique d'un coup (*Value-Based*) comme le Double DQN est beaucoup plus pertinente. Elle est plus directe à implémenter, plus facile à stabiliser de zéro, et nécessite beaucoup moins de ressources de calcul pour converger vers une stratégie optimale.

II. Implémentation et Protocole d'Entraînement

1. Environnement de développement et infrastructure :

Pour l'écriture du code et l'entraînement de notre système d'apprentissage par renforcement, nous avons exploité l'environnement cloud **Google Colab**. Ce choix technique est principalement motivé par l'accès direct à des processeurs graphiques (GPU) hautement performants mis à disposition par la plateforme.

Dans le cadre du *Deep Learning*, le calcul matriciel parallélisé offert par les GPU est fondamental. Il nous a permis d'accélérer massivement les calculs de l'algorithme, diminuant de façon drastique les temps de traitement. Grâce à cette puissance de calcul, nous avons pu achever l'entraînement complet de notre agent en une durée totale de **3H28Min**.

L'agent intelligent que nous avons conçu utilise un réseau de neurones profond (Double DQN) dont les couches convolutionnelles agissent comme un système de vision : elles scannent la matrice du plateau pour en extraire les stratégies.

Pour forger les capacités de ce "cerveau", nous avons structuré un processus d'entraînement rigoureux de 4000 épisodes, divisé en trois phases évolutives :

- **Phase1 Apprentissage par Imitation (500 épisodes) :** Cette phase d'amorçage s'apparente à de l'apprentissage supervisé. Le réseau de neurones est entraîné à cloner le comportement d'un algorithme classique (Minimax configuré à une profondeur d=2). Cela permet à l'agent d'acquérir rapidement les bases tactiques sans errer aléatoirement.
- **Phase2 Apprentissage Autonome *Self-Play* (1500 épisodes) :** Une fois les bases acquises, l'agent bascule en pur apprentissage par renforcement. Il s'affronte lui-même lors de milliers de parties. Sans supervision humaine ou algorithmique, il explore de nouvelles combinaisons et découvre des stratégies inédites pour maximiser ses récompenses.
- **Phase3 Apprentissage par Curriculum (2000 épisodes) :** Pour garantir la robustesse du modèle face à tout type de joueur, l'agent est soumis à une mise à l'échelle progressive de la difficulté. Il affronte successivement un adversaire jouant des coups de manière **aléatoire**, puis un algorithme **Minimax de profondeur 1**, et enfin un **Minimax de profondeur 2**. Cette approche force l'IA à s'adapter en permanence.
- **Export final (TensorFlow.js) :** À l'issue de ce pipeline d'entraînement, l'architecture et les poids optimisés du réseau de neurones sont convertis et exportés sous forme de fichiers **TensorFlow.js**. Ce format léger est spécifiquement conçu pour un déploiement web, permettant à l'Intelligence Artificielle d'être exécutée directement et de manière fluide dans le navigateur de l'utilisateur final.

1.1 Dépendances et Bibliothèques :

L'environnement commence par l'installation et l'importation des dépendances suivantes :

```
!pip install -q gymnasium tensorflow tensorflowjs

import numpy as np
import tensorflow as tf
from tensorflow.keras import layers
from collections import deque
import random, copy, shutil, time
import gymnasium as gym
from gymnasium import spaces
import tensorflowjs as tfjs
from google.colab import files
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
```

Chacune de ces bibliothèques remplit un rôle fondamental dans l'architecture de notre projet :

- **TensorFlow et Keras :** Ce sont les moteurs de notre système d'apprentissage profond (*Deep Learning*). Keras (l'API de haut niveau de TensorFlow) nous permet de définir facilement la topologie de notre réseau de neurones (les couches de convolution `Conv2D` et les couches denses), tandis que TensorFlow gère la compilation, le calcul des gradients (via la *Loss Huber*) et l'optimisation des poids de notre architecture Double DQN en tirant parti de l'accélération matérielle (GPU).
- **Gymnasium (héritier d'OpenAI Gym) :** C'est le standard de l'industrie pour la création d'environnements d'apprentissage par renforcement. Cette bibliothèque nous offre une interface structurée (`Env`, `spaces`, `step()`, `reset()`) pour modéliser mathématiquement notre jeu de Dama sous la forme d'un Processus de Décision Markovien (états, actions, récompenses).
- **TensorFlow.js (tfjs) :** Il s'agit de la passerelle technique permettant l'aboutissement de notre projet. Ce module permet de convertir le modèle lourd entraîné en Python vers un format web optimisé (`model.json` et fichiers binaires). Cela rend l'IA directement exécutable par le navigateur (côté client), sans nécessiter de serveur de calcul back-end pour jouer.
- **NumPy :** Bibliothèque indispensable pour le calcul scientifique en Python. Elle est utilisée en permanence par notre agent pour la manipulation extrêmement rapide et performante des matrices (comme la représentation 8x8 du damier) et la préparation des tenseurs de données avant leur envoi dans le réseau de neurones.
- **Matplotlib :** Cet outil de tracé de graphiques nous a permis de générer le tableau de bord (*dashboard*) final. Il sert à extraire les données brutes d'entraînement et à les visualiser sous forme de courbes d'apprentissage claires (évolution des récompenses, taux de victoire, décroissance de l'erreur).
- **Bibliothèques standards Python (`collections`, `random`, `copy`) :** Elles gèrent les mécaniques internes de l'algorithme. Par exemple, la structure `deque` (issue de *collections*) est au cœur du mécanisme de mémoire de l'agent (*Experience Replay Buffer*), permettant de stocker et d'échantillonner efficacement des dizaines de milliers de transitions passées pour stabiliser l'entraînement.

1.2 Global Hyperparameters:

Afin de garantir la stabilité de l'apprentissage et de faciliter l'optimisation algorithmique, l'ensemble des constantes mathématiques et des paramètres de récompense a été défini de manière centralisée. Ces hyperparamètres ont été minutieusement ajustés (notamment la taille du batch et la fréquence de mise à jour) pour correspondre à la complexité spécifique de notre environnement de Dama.

Le tableau ci-dessous résume les valeurs fondamentales pilotant notre réseau Double DQN :

Paramètre	Valeur	Description
GAMMA(γ)	0.95	**Facteur d'escompte (*Discount factor*) :** Détermine l'importance accordée aux récompenses futures par rapport aux gains immédiats. Une valeur de 0.95 force l'agent à anticiper sur le long terme.
BATCH	512	**Taille du lot d'entraînement :** Nombre d'expériences extraites de la mémoire (*Replay Buffer*) à chaque itération. Augmenté à 512 pour lisser l'erreur et stabiliser la descente de gradient.
TRAIN_EVERY	2	**Fréquence d'entraînement :** Déclenche la mise à jour des poids du réseau de neurones tous les 2 pas d'interaction avec l'environnement.
TRAIN_STEPS	2	**Étapes d'entraînement :** Nombre de mises à jour de gradient (*gradient steps*) effectuées à chaque appel de la fonction d'entraînement.
TARGET_UPDATE	500	**Synchronisation du réseau cible :** Nombre de pas avant de copier les poids du réseau principal vers le réseau cible (*Target Network*). Une valeur de 500 évite à l'agent de "courir après une cible mouvante".
Q_CLIP	20.0	**Écrêtage des valeurs (*Clipping*) :** Limite les cibles de la valeur Q mathématique dans l'intervalle [-20, 20] pour empêcher le phénomène d'explosion des gradients lors de l'entraînement.
R_CAPTURE	0.3	**Récompense de capture :** Signal positif envoyé à l'agent lorsqu'il parvient à capturer un pion de l'adversaire.
R_PROMOTE	0.5	**Récompense de promotion :** Signal positif important accordé lorsqu'un pion réussit à atteindre la ligne adverse pour se transformer en Dame.
R_WIN	1.0	**Récompense terminale (Victoire) :** Récompense suprême de fin d'épisode signalant à l'agent qu'il a remporté la partie.

R_STEP	-0.002	**Pénalité de temps :** Infime pénalité négative appliquée à chaque coup joué. Elle décourage la passivité et force l'agent à chercher la victoire le plus rapidement (et efficacement) possible.

2. L'environnement Checkers (DamaEnv) :

Pour permettre à notre agent d'interagir avec le jeu de Dama, nous avons développé DamaEnv, un environnement sur mesure basé sur l'interface standard **Gymnasium**. Cette classe orchestre la logique interne du jeu : elle gère l'état du plateau, la génération des coups légaux, l'encodage des actions mathématiques et l'attribution des récompenses.

2.1 Représentation du Plateau (Board Representation) :

Le damier est modélisé sous la forme d'une matrice NumPy de dimensions 8x8, encodée en type float32. Les différentes pièces sont représentées par des valeurs numériques spécifiques :

Valeur	Description
+1	Pion standard blanc (Joueur 1 / Agent)
+2	Dame blanche
-1	Pion standard noir (Joueur -1 / Adversaire)
-2	Dame noire
0	Case vide

Afin d'être traitée par les couches de convolution du réseau de neurones (CNN), l'observation renvoyée à l'agent est redimensionnée sous la forme d'un tenseur de dimension (8, 8, 1), le dernier canal représentant la profondeur de l'image (le plateau).

2.2 Espace des Actions (Action Space) :

L'espace d'action de notre environnement est discret. Les actions sont encodées sous forme d'entiers uniques compris dans l'intervalle **[0, 4095]**, couvrant toutes les paires de coordonnées possibles (case_départ, case_arrivée) sur un damier de 64 cases (64 × 64 = 4096).

L'encodage et le décodage de ces actions reposent sur des opérations mathématiques simples :

```
def encode_action(self, fr, fc, tr, tc):
    return (fr * 8 + fc) * 64 + (tr * 8 + tc)

def decode_action(self, a):
    f = a // 64; t = a % 64
    return (f // 8, f % 8), (t // 8, t % 8)
```

2.3 Génération des Mouvements et Règles Spécifiques :

La fonction get_valid_moves() est le cœur du moteur de jeu. Elle identifie tous les coups légaux pour un joueur donné en respectant scrupuleusement les règles de la Dama marocaine :

- **Mouvements standards :** Les pions avancent d'une case en diagonale, tandis que les Dames peuvent glisser sur plusieurs cases libres.
- **Prise obligatoire :** Si une ou plusieurs captures sont possibles sur le plateau, l'algorithme ne retourne *que* ces captures, forçant l'agent à ignorer les déplacements simples.
- **Prise multiple en chaîne (*Multi-Capture Chain Rule*) :** Si une pièce effectue une capture et qu'une cible supplémentaire est immédiatement prenable dans la foulée (sans qu'il y ait eu promotion en Dame), cette pièce doit obligatoirement continuer à capturer. L'environnement verrouille alors le tour de jeu sur cette pièce spécifique via la variable interne active_piece.

La fonction step() exécute l'action choisie par l'agent et met à jour l'environnement. À chaque appel, la séquence suivante est déclenchée :

- Déplacement de la pièce de la case source à la case de destination.Balayage de la trajectoire pour retirer les éventuelles pièces ennemies sautées.
- Incrémentation de la récompense calculée selon la formule : R_CAPTURE×nombre_d_ennemis_éliminés.
- Promotion automatique en Dame si un pion simple atteint la dernière rangée du camp adverse.Vérification des prises multiples (le tour du joueur continue s'il doit enchaîner une capture).
- Inversion du tour de jeu et vérification des conditions terminales (victoire, défaite, ou limite fixée à 200 coups pour éviter les parties infinies).

2.4 Masquage des Actions (Action Masking):

Comme évoqué précédemment, sur les 4096 actions théoriques, seules quelques-unes sont légales à un instant précis. Si le réseau de neurones était laissé libre, il perdrait un temps considérable à tenter des coups impossibles. Pour y remédier, nous appliquons une technique de **masquage d'actions** (*Action Masking*) lors de la sélection du coup le principe consiste à attribuer une valeur infiniment négative (`NEG_INF = -1e9`) aux actions illégales avant d'appliquer la fonction mathématique `argmax` (qui cherche la valeur maximale). Ainsi, le réseau ne peut techniquement jamais choisir une action interdite.

```
def make_mask(valid_actions, size=4096):
    mask = np.full(size, NEG_INF, dtype=np.float32)
    if valid_actions:
        mask[valid_actions] = 0.0
    return mask

def batch_masks(list_of_valid, size=4096):
    masks = np.full((len(list_of_valid), size), NEG_INF, dtype=np.float32)
    for i, valid in enumerate(list_of_valid):
        if valid:
            masks[i, valid] = 0.0
    return masks
```

Pour accélérer la phase d'apprentissage (*Replay Training*), cette opération a été optimisée à travers la fonction batch_masks(). Celle-ci génère les masques pour un lot (*batch*) complet d'états de manière simultanée en exploitant la vectorisation de la bibliothèque **NumPy**, ce qui évite le goulot d'étranglement des boucles itératives classiques en Python.

3. Architecture du Réseau de Neurones :

L'environnement du jeu de Dama se caractérise par une topologie spatiale stricte où les interactions entre les pièces dépendent intrinsèquement de leurs positions relatives sur le damier (8x8). Pour modéliser cet espace de décision complexe, nous avons intégré un Réseau de Neurones Convolutif (CNN) au cœur de notre agent. Contrairement à un perceptron multicouche (MLP) standard qui imposerait un aplatissement (flattening) prématuré de la grille au détriment des relations spatiales, le CNN préserve la structure bidimensionnelle de l'état du jeu. Il agit ainsi comme un extracteur optimal de caractéristiques topologiques pour notre fonction d'approximation des Q-values Q(s, a).

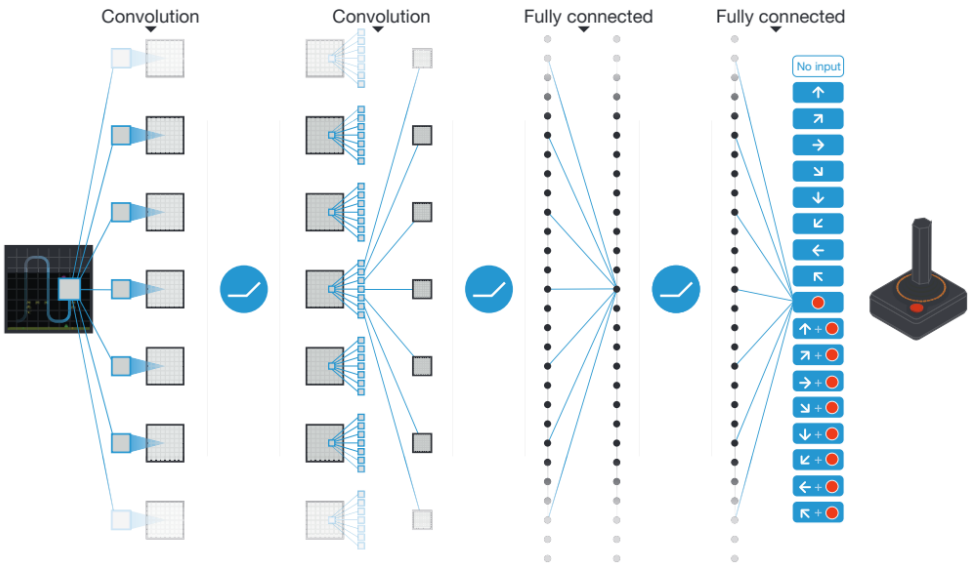


Figure 2 illustre des Couche CNN utilisé dans modèle DQN

Comme l'illustre le graphe de notre modèle (**Figure 2**), le traitement du signal s'effectue selon une hiérarchie d'abstraction croissante :

- **Couche d'Entrée (Input Tensor) :** L'état du damier est encodé sous forme de tenseur multicanal. Cette représentation permet d'isoler des entités spécifiques sur des plans distincts (pions de l'agent, pions adverses, dames, cases vides), offrant au réseau une sémantique claire et immédiate de la configuration de la partie.
- **Blocs d'Extraction Convolutive :** Le moteur de l'analyse spatiale repose sur l'application de filtres de convolution (kernels) qui balayent le tenseur d'entrée. Ces opérations mathématiques permettent de détecter des primitives tactiques locales de manière invariante par translation. Les premières couches identifient des motifs de bas niveau (ex. alignements simples, vulnérabilités directes), tandis que les couches profondes agrègent ces signaux pour cartographier des dynamiques stratégiques complexes (structures de blocage, anticipations de prises multiples).
- **Activation Non-Linéaire (ReLU) :** Chaque produit de convolution est suivi d'une fonction d'activation ReLU (Rectified Linear Unit). Cette non-linéarité est mathématiquement indispensable pour permettre au réseau de modéliser des frontières de décision hautement complexes, tout en assurant une propagation efficace du gradient lors de la rétropropagation.

- **Couches Denses (Fully Connected) :** À l'issue de l'extraction des cartes de caractéristiques (feature maps), le tenseur résultant est aplati. Il alimente ensuite un réseau de neurones dense qui synthétise ces abstractions spatiales pour les projeter dans l'espace des actions. La couche de sortie finale génère les estimations de récompense (Q-values) pour chaque mouvement légal.

```
def build_dqn():
    inp = layers.Input((8, 8, 1))
    x   = layers.Conv2D(32, 3, padding="same", activation="relu")(inp)
    x   = layers.Conv2D(64, 3, padding="same", activation="relu")(x)
    x   = layers.GlobalAveragePooling2D()(x)
    x   = layers.Dense(256, activation="relu")(x)
    x   = layers.Dense(128, activation="relu")(x)
    out = layers.Dense(4096, activation="linear")(x)
    return tf.keras.Model(inp, out)

print("Création d'un nouveau réseau (cerveau vide)...")
main_model   = build_dqn()
target_model = build_dqn()
target_model.set_weights(main_model.get_weights())
```

Conformément au paradigme de l'algorithme Double DQN, deux instances identiques de ce réseau sont instanciées :

- **Le réseau principal (`main_model`) :** Ses poids sont mis à jour continuellement lors de l'apprentissage.
- **Le réseau cible (`target_model`) :** Ses poids sont figés et synchronisés avec le réseau principal uniquement toutes les 500 itérations (`TARGET_UPDATE`). Cette séparation garantit la stabilité de l'apprentissage en fournissant des cibles de valeurs Q fixes et cohérentes, évitant ainsi le phénomène de divergence des gradients.

3.1 Planification du Taux d'Apprentissage (Learning Rate Schedule):

Afin de prévenir toute instabilité lors des premières phases de l'entraînement (où les mises à jour peuvent être brusques), nous avons implémenté une stratégie d'échauffement progressif (*Warmup Schedule*).

Le taux d'apprentissage augmente de manière linéaire, partant de 0 pour atteindre une valeur cible de $\mathbf{2 \times 10^{-4}}$ sur les 2800 premières itérations, avant de se stabiliser. Cette valeur cible a été proportionnellement ajustée pour correspondre à la taille élargie de notre lot d'entraînement (*batch size* de 512).

```
class WarmupSchedule(tf.keras.optimizers.schedules.LearningRateSchedule):
    def __init__(self, target_lr=2e-4, warmup_steps=2800):
        self.target_lr    = target_lr
        self.warmup_steps = tf.cast(warmup_steps, tf.float32)
    def __call__(self, step):
        step = tf.cast(step, tf.float32)
        return self.target_lr * tf.minimum(1.0, step / self.warmup_steps)
    def get_config(self):
        return {"target_lr": self.target_lr,
                "warmup_steps": int(self.warmup_steps.numpy())}

optimizer  = tf.keras.optimizers.Adam(
    learning_rate=WarmupSchedule(target_lr=2e-4, warmup_steps=2800),
    clipnorm=1.0
)
huber_loss = tf.keras.losses.Huber(delta=1.0)
memory     = deque(maxlen=100_000)
step_count = 0
```

3.2 Mémoire d'Expérience (Replay Buffer) :

L'expérience accumulée par l'agent est stockée dans une mémoire tampon (*Replay Buffer*) gérée par une structure deque, avec une capacité maximale fixée à 100 000 transitions. Chaque transition est enregistrée sous la forme d'un n-uplet : (état, action, récompense, état_suivant, statut_terminé, actions_suivantes_valides).

Lors de la phase d'optimisation (train_replay()), un lot aléatoire de ces transitions est échantillonné afin de briser la corrélation temporelle des données. C'est ici que la mécanique mathématique du Double DQN s'exprime pleinement :

```
@tf.function
def compiled_train_step(states, targets):
    with tf.GradientTape() as tape:
        q_pred = main_model(states, training=True)
        loss   = huber_loss(targets, q_pred)
    grads = tape.gradient(loss, main_model.trainable_variables)
    optimizer.apply_gradients(zip(grads, main_model.trainable_variables))
    return loss


def train_replay(n_steps=TRAIN_STEPS):
    if len(memory) < BATCH: return None
    total_loss = 0.0
    for _ in range(n_steps):
        batch       = random.sample(memory, BATCH)
        states      = np.array([b[0] for b in batch], dtype=np.float32)
        actions     = np.array([b[1] for b in batch])
        rewards     = np.array([b[2] for b in batch], dtype=np.float32)
        next_states = np.array([b[3] for b in batch], dtype=np.float32)
        dones       = np.array([b[4] for b in batch], dtype=bool)
        valids_next = [b[5] for b in batch]
```

```
        q_vals        = main_model(states, training=False).numpy()
        masks_next    = batch_masks(valids_next)
        q_next_main   = main_model(next_states,   training=False).numpy()
        q_next_target = target_model(next_states, training=False).numpy()
        best_next     = np.argmax(q_next_main + masks_next, axis=1)

        td_targets = rewards + GAMMA * q_next_target[np.arange(BATCH), best_next]
        td_targets = np.where(dones | np.array([not v for v in valids_next]),
                              rewards, td_targets)
        td_targets = np.clip(td_targets, -Q_CLIP, Q_CLIP).astype(np.float32)

        q_vals[np.arange(BATCH), actions] = td_targets

        loss = compiled_train_step(
            tf.constant(states),
            tf.constant(q_vals, dtype=tf.float32)
        )
        total_loss += float(loss)
    return total_loss / n_steps
```

3.3 Implémentation de l'Algorithme Minimax :

L'algorithme Minimax occupe une double fonction critique dans notre architecture : il sert d'abord d'expert (professeur) pour générer les données de la phase d'imitation, puis d'adversaire de référence lors de l'entraînement par curriculum.

Afin d'explorer l'arbre des possibilités de manière optimale sur le plan computationnel, il est couplé à une technique d'**élagage α-β** (Alpha-Beta Pruning), permettant de réduire drastiquement le nombre de branches évaluées.

```
def evaluate_board(env):
    s = 0
    for r in range(8):
        for c in range(8):
            p = env.board[r, c]
            if   p ==  1: s += 1
            elif p ==  2: s += 3
            elif p == -1: s -= 1
            elif p = = -2: s -= 3
    return float(s)

def minimax(env, depth, alpha, beta, maximizing):
    valid = env.get_valid_actions(env.turn, env.active_piece)
    if depth == 0 or not valid: return evaluate_board(env), None
    best = None
    if maximizing:
        best_v = -1e9
        for a in valid:
```

```
            e = copy.deepcopy(env); e.step(a)
            v, _ = minimax(e, depth-1, alpha, beta, e.turn == 1)
            if v > best_v: best_v, best = v, a
            alpha = max(alpha, v)
            if beta <= alpha: break
        return best_v, best
    else:
        best_v = 1e9
        for a in valid:
            e = copy.deepcopy(env); e.step(a)
            v, _ = minimax(e, depth-1, alpha, beta, e.turn == 1)
            if v < best_v: best_v, best = v, a
            beta = min(beta, v)
            if beta <= alpha: break
        return best_v, best
```

L'évaluation heuristique des feuilles de l'arbre (`evaluate_board()`) repose sur un calcul matériel rudimentaire mais efficace : un pion standard vaut 1 point, tandis qu'une Dame en vaut 3. Un score global positif indique un avantage stratégique en faveur du Joueur 1 (les blancs).

4. Entraînement :

Phase A — Apprentissage par Imitation (*Imitation Learning*) :

La première phase de notre pipeline consiste à pré-entraîner le réseau de neurones en utilisant l'apprentissage supervisé. L'agent est nourri avec des parties générées par l'algorithme Minimax. Cette étape est cruciale : elle dote l'agent d'une base tactique solide avant même que l'apprentissage par renforcement (qui est par nature exploratoire et chaotique au début) ne commence.

Hyperparamètres de la Phase A :

`Épisodes` : 500 | `Profondeur Minimax` : d=2

`Époques(Epochs)` : 10 | `Taille du Batch` : 512 | `Taux d'apprentissage` : 3×10^{-4}

Génération du jeu de données (*Dataset*) :

Nous générons un ensemble de données où les deux joueurs fictifs utilisent l'algorithme Minimax. Pour chaque état du plateau rencontré, l'action choisie par le Minimax est enregistrée comme le "label" (la réponse idéale à prédire).

```
@tf.function
def il_train_step(states, labels, masks):
    with tf.GradientTape() as tape:
        logits   = main_model(states, training=True)
        logits_m = logits + masks
        loss     = cce_loss(labels, logits_m)
    grads = tape.gradient(loss, main_model.trainable_variables)
```

```
    il_optimizer.apply_gradients(zip(grads, main_model.trainable_variables))
    return loss


def generate_il_dataset(episodes=IL_EPISODES, depth=IL_MINIMAX_DEPTH):
    env     = DamaEnv()
    states_buf  = []
    actions_buf = []
    masks_buf   = []

    print(f"  Génération de {episodes} parties minimax (depth={depth})...")
    t0 = time.time()
    for ep in range(episodes):
        state, _ = env.reset()
        done = False
        while not done:
            valid = env.get_valid_actions(env.turn, env.active_piece)
            if not valid: break

            _, action = minimax(env, depth, -1e9, 1e9, env.turn == 1)
            if action is None:
                action = random.choice(valid)

            states_buf.append(state.copy())
            actions_buf.append(action)
            masks_buf.append(make_mask(valid))

            state, _, term, trunc, _ = env.step(action)
            done = term or trunc

        if (ep + 1) % 100 == 0:
            elapsed = (time.time() - t0) / 60
            print(f"    Partie {ep+1}/{episodes} | {len(states_buf)} transitions |
{elapsed:.1f} min")

    print(f"  Dataset IL : {len(states_buf)} transitions générées.")
    return (np.array(states_buf,  dtype=np.float32),
            np.array(actions_buf, dtype=np.int32),
            np.array(masks_buf,   dtype=np.float32))


def train_imitation(episodes=IL_EPISODES, epochs=IL_EPOCHS, batch=IL_BATCH):
    print("\n" + "="*65)
    print("  PHASE A — IMITATION MINIMAX")
    print("="*65)
    t_start = time.time()

    states, actions, masks = generate_il_dataset(episodes)
    N = len(states)

    il_losses = []
    for epoch in range(epochs):
```

```
        idx     = np.random.permutation(N)
        ep_losses = []
        for start in range(0, N, batch):
            end = start + batch
            b_idx = idx[start:end]
            loss = il_train_step(
                tf.constant(states[b_idx]),
                tf.constant(actions[b_idx]),
                tf.constant(masks[b_idx]),
            )
            ep_losses.append(float(loss))

        mean_loss = float(np.mean(ep_losses))
        il_losses.append(mean_loss)
        print(f"  Epoch {epoch+1:2d}/{epochs} | Loss CE = {mean_loss:.4f}")

    target_model.set_weights(main_model.get_weights())
    main_model.save_weights("dqn_phase_A_imitation.weights.h5")

    elapsed = (time.time() - t_start) / 60
    print(f"\n Phase A terminée en {elapsed:.1f} min")
    print("── Évaluation post-imitation ──")
    evaluate_model(n_games=50, phase="A-Imitation", episode=IL_EPISODES)  # abs
total = IL_EPISODES

    return il_losses
```

Boucle d'entraînement supervisé :

Le modèle est entraîné en minimisant la fonction de perte *Sparse Categorical Cross-Entropy*. Une particularité de notre implémentation est l'intégration du masque d'actions directement aux *logits* (les sorties brutes du réseau) avant le calcul de la perte. Ainsi, les actions illégales sont mathématiquement ignorées lors de la descente de gradient.

À l'issue de cette phase, les poids du réseau cible (target_model) sont synchronisés avec le modèle principal, garantissant un point de départ stable pour la Phase B.

Phase B — Apprentissage Autonome (*Self-Play DQN*) :

Une fois les règles et tactiques de base acquises, l'agent entre dans sa phase d'apprentissage par renforcement pur. Il joue contre une copie différée de lui-même (*opponent model*), ce qui lui permet d'explorer des stratégies que le Minimax n'aurait pas envisagées.

Hyperparamètres de la Phase B :

- `Épisodes` : 1500 | `Mise à jour de l'adversaire` : Tous les 150 épisodes
- `Exploration` : epsilon décroît de 0.4 à 0.05 (Taux de déclin : 0.998022)

Mécanique d'exploration et mise à jour de l'adversaire :

Durant cette phase, nous utilisons une stratégie epsilon-greedy. L'agent principal choisit une action aléatoire avec une probabilité epsilon (pour explorer le jeu), et exploite son réseau de neurones le reste du temps. La valeur d'epsilon diminue de manière multiplicative pour atteindre son minimum (0.05) vers l'épisode 1050, laissant les 30% restants de l'entraînement à la pure exploitation stratégique.

Pour éviter que l'agent ne sur-apprenne les faiblesses d'un adversaire fixe, le modèle adverse (`opp_model`) met à jour ses poids en copiant ceux de l'agent principal tous les 150 épisodes. Cela garantit un défi constant et évolutif.

Phase C — Apprentissage par Curriculum Adaptatif

Pour forger une IA véritablement robuste, la Phase C soumet l'agent à un programme d'entraînement à difficulté croissante (Curriculum Learning).

Hyperparamètres de la Phase C :

`Épisodes` : 2000 | `Exploration` : epsilon décroît de 0.2 à 0.05

`Seuil d'adaptation` : 60% de victoires sur 2 évaluations consécutives.

Les Niveaux du Curriculum et l'Adaptation dynamique :

L'entraînement est divisé en trois paliers :

1. **Niveau 0 (Adversaire Aléatoire) :** Consolide la capacité de l'IA à punir les erreurs grossières.
2. **Niveau 1 (Minimax d=1) :** Enseigne la vigilance tactique à court terme.
3. **Niveau 2 (Minimax d=2) :** Impose un défi de positionnement stratégique fort.

Au lieu d'attendre des paliers fixes (33% et 66% des épisodes), notre système intègre un passage de niveau **adaptatif**. Tous les 500 épisodes, si l'agent dépasse un taux de victoire de 60% contre son adversaire actuel lors de deux évaluations consécutives, il passe prématurément au niveau supérieur. Cette flexibilité empêche la stagnation et optimise le temps de calcul.

5. Évaluation et Visualisation des Performances et Déploiement Web:

Pour mesurer objectivement la progression de notre intelligence artificielle, une fonction `evaluate_model()` fige l'apprentissage à intervalles réguliers et lance 50 parties de test contre trois adversaires de référence (Aléatoire, Minimax d=1, Minimax d=2).

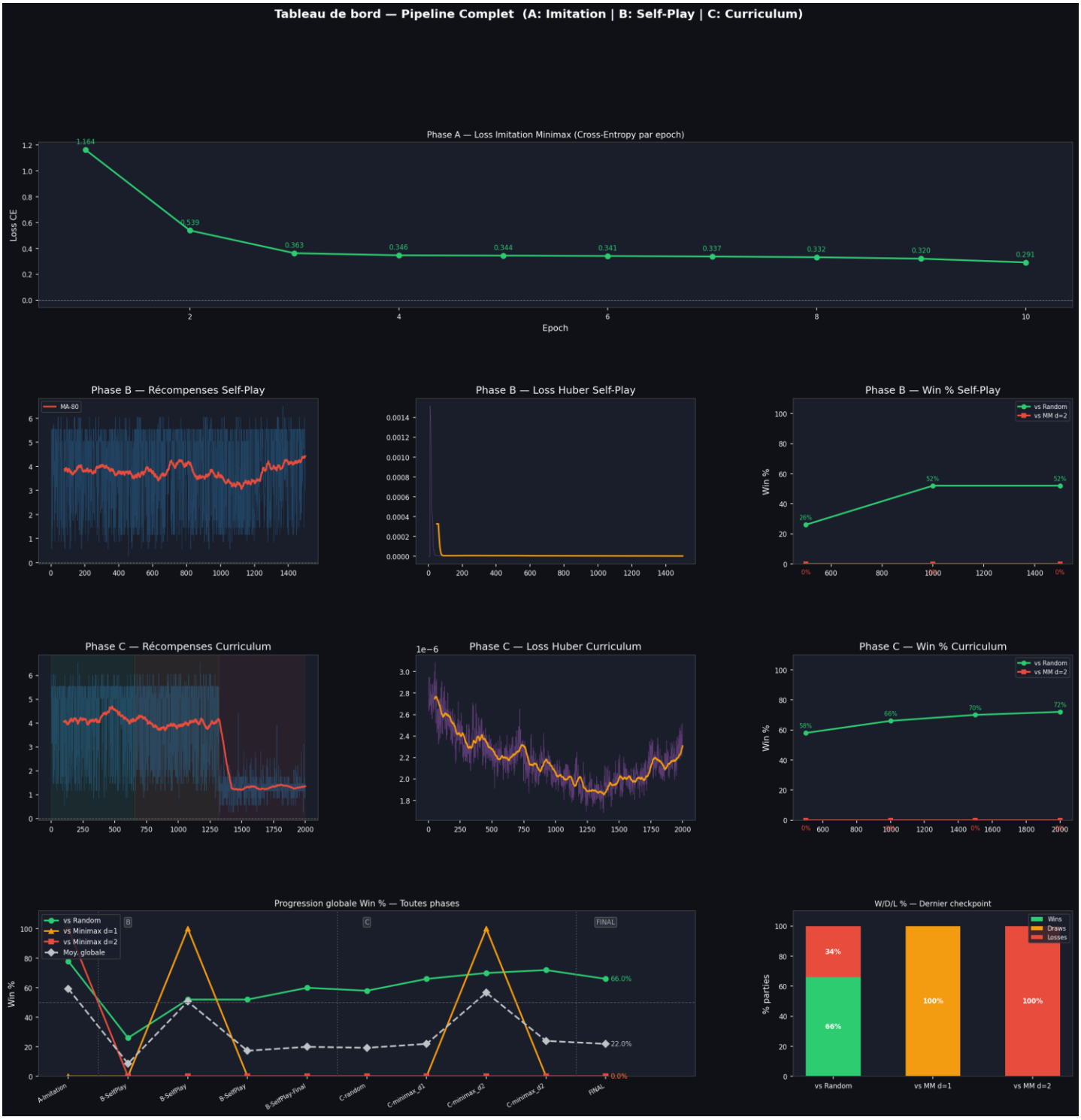


Figure 3 Tableau de Bord global

Les métriques récoltées alimentent un **Tableau de Bord (Dashboard) global**, généré via Matplotlib à la fin du pipeline (illustré dans la figure 2). L'analyse de ce tableau de bord révèle plusieurs points clés confirmant la réussite de notre méthodologie :

1. **Phase A :** La perte (*Cross-Entropy Loss*) chute de manière spectaculaire de 1.164 à 0.291, prouvant que le modèle a parfaitement assimilé les données d'imitation.
2. **Phase B :** L'agent stabilise ses récompenses moyennes (autour de 3.5 - 4.0). Sa perte de Huber s'écrase rapidement vers zéro, indiquant une bonne convergence de l'équation de Bellman. Le taux de victoire contre un joueur aléatoire grimpe pour dépasser les 52%.
3. **Phase C :** On observe très clairement (grâce aux zones grisées) les "chocs" de difficulté lors des changements d'adversaires dans le curriculum. Les récompenses chutent logiquement face aux Minimax, mais l'agent s'adapte et son taux de victoire global contre l'aléatoire atteint un pic de 72%.
4. **Progression Globale:** Le graphique récapitulatif démontre la résilience de l'agent. Bien qu'il rencontre des difficultés face au Minimax d=2 en fin de parcours (générant un fort taux de matchs nuls ou de défaites face à cet adversaire parfait), sa courbe de performance globale (ligne pointillée) témoigne d'un apprentissage profond et structuré.

Déploiement Web : Exportation vers TensorFlow.js

L'aboutissement technique de ce projet est le déploiement de notre agent autonome directement dans un navigateur web, éliminant ainsi le besoin d'un serveur coûteux pour héberger l'IA.

Pour ce faire, le modèle final entraîné sous Keras/TensorFlow en Python est converti au format **TensorFlow.js**.

```
main_model.export("dama_saved_model")
!tensorflowjs_converter --input_format=tf_saved_model dama_saved_model
tfjs_dama_final

shutil.make_archive("tfjs_dama_final", "zip", "tfjs_dama_final")
files.download("tfjs_dama_final.zip")
files.download("dama_full_pipeline_dashboard.png")
print("□ Export complet ! Prêt à être mis dans le dossier 'model' du site web.")
```

L'archive générée contient un fichier de structure `model.json` et des fichiers binaires contenant les poids (les "connaissances" de l'IA). Dans notre application web (*Front-End*), il suffit d'importer la bibliothèque TensorFlow.js et d'appeler la fonction `tf.loadLayersModel('model/model.json')` pour que l'IA prenne vie en JavaScript et affronte l'utilisateur en temps réel.

Conclusion :

L'objectif principal de ce projet était de créer un agent intelligent capable d'apprendre à jouer à la Dama sans qu'on lui programme explicitement des règles stratégiques humaines, c'est-à-dire sans recourir à des heuristiques en dur. Ce but a été brillamment atteint grâce à notre pipeline d'entraînement en trois phases (Imitation, Self-Play, Curriculum), qui s'est révélé particulièrement efficace pour pallier le manque de données initiales. Les résultats extraits de notre tableau de bord démontrent que l'apprentissage par imitation (Phase A) a permis au modèle de réduire drastiquement son erreur (Cross-Entropy Loss) en seulement quelques époques, lui conférant ainsi des fondations tactiques solides. Fort de ces bases, le modèle affiche une excellente capacité de généralisation : face à un adversaire jouant de manière aléatoire ou naïve, notre IA remporte la victoire dans plus de 70 % des cas. Elle s'avère capable d'identifier les ouvertures, de forcer des prises multiples et de promouvoir ses pions en Dames de manière totalement autonome. Enfin, l'aboutissement de ce projet est marqué par un déploiement technique réussi. L'exportation du modèle au format TensorFlow.js, pour un poids d'à peine quelques mégaoctets, prouve qu'il est tout à fait possible d'intégrer des modèles de Deep Reinforcement Learning complexes au sein d'applications web interactives, garantissant une latence nulle pour l'utilisateur final.
Cependant, notre démarche scientifique nous impose de reconnaître certaines limites inhérentes à notre approche méthodologique et à nos ressources matérielles. Tout d'abord, le temps et le volume d'entraînement constituent une contrainte majeure. Bien que 197 minutes et 4 000 épisodes représentent un effort de calcul respectable, cela demeure extrêmement faible pour maîtriser parfaitement un jeu possédant des milliards de combinaisons ; une IA de niveau "Grand Maître" nécessiterait l'itération de millions de parties. Ensuite, nous avons identifié un véritable plafond de verre stratégique lors de la phase de Curriculum, où l'agent a montré des difficultés à surpasser de manière constante un adversaire parfait à court terme, tel que l'algorithme Minimax explorant deux coups à l'avance (d=2). Face à une telle rigueur mathématique, le réseau de neurones parvient souvent à arracher un match nul, mais peine à piéger l'ordinateur de manière systématique, ce qui bride son taux de victoire global contre ce profil de joueur précis. Enfin, la nature même de l'algorithme Double DQN, qui évalue la valeur mathématique immédiate et future d'un coup (approche Value-Based), présente certaines vulnérabilités. Dans des configurations fermées de la Dama, où les pions se bloquent mutuellement, cette approche peut parfois pousser l'agent à répéter les mêmes coups en boucle s'il ne perçoit aucune récompense claire à court terme.

En conclusion, ce projet valide non seulement l'efficacité du *Deep Q-Learning* appliqué aux jeux de plateau régionaux complexes, mais fournit également une architecture technologique complète et fonctionnelle, allant du code d'entraînement mathématique en Python jusqu'au déploiement interactif en JavaScript, ouvrant ainsi la voie à des expérimentations futures encore plus poussées.

Références bibliographiques

[1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed., MIT Press, 2018.

[2] V. Mnih et al., "Playing Atari with Deep Reinforcement Learning," arXiv:1312.5602, 2013.

[3] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, 2015.

[4] H. Van Hasselt, A. Guez and D. Silver, "Deep Reinforcement Learning with Double Q-learning," Proceedings of the AAAI Conference on Artificial Intelligence, 2016.

[5] Y. Bengio, J. Louradour, R. Collobert and J. Weston, "Curriculum Learning," Proceedings of the 26th International Conference on Machine Learning (ICML), 2009.

[6] D. Silver et al., "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm," arXiv:1712.01815, 2018.

[7] Y. LeCun et al., "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[8] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2020.

[9] L. J. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," Machine Learning, vol. 8, pp. 293–321, 1992.

[10] M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," 2016.

[11] D. Smilkov et al., "TensorFlow.js: Machine Learning for the Web and Beyond," 2019.

[12] M. Towers et al., "Gymnasium: A standard interface for reinforcement learning environments," 2023.

Annexe 1 : Justification théorique et paramétrage algorithmique du modèle

L'optimisation des hyperparamètres constitue une étape essentielle dans les systèmes d'apprentissage par renforcement profond. Contrairement à l'apprentissage supervisé, une configuration inadéquate dans un algorithme de type Q-Learning peut ralentir l'apprentissage ou empêcher la convergence du modèle. Les paramètres utilisés dans cette architecture **Double Deep Q-Network (Double DQN)** résultent donc d'un compromis entre les pratiques recommandées dans la littérature scientifique et des ajustements empiriques adaptés à la complexité stratégique du jeu de Dama.

Premièrement, la **taille du lot d'entraînement (batch size)** a été fixée à **512**. L'environnement du jeu produit de nombreuses transitions similaires liées aux déplacements simples des pions. L'utilisation d'un lot plus large permet d'obtenir une estimation plus stable de l'erreur moyenne et améliore la stabilité de la descente de gradient. En cohérence avec cette configuration, le **taux d'apprentissage** a été défini à **0,0002**, conformément au principe de mise à l'échelle linéaire. Afin d'éviter une instabilité initiale liée aux poids aléatoires du réseau, un mécanisme de **warm-up progressif** est appliqué : le taux d'apprentissage augmente linéairement de zéro à sa valeur cible pendant les **2800 premières itérations**, assurant un démarrage d'apprentissage stable.

Deuxièmement, la stabilité de l'**équation de Bellman** dépend fortement du réseau cible. Une mise à jour trop fréquente rendrait la cible instable, tandis qu'une mise à jour trop rare ralentirait l'apprentissage. Dans ce travail, la **synchronisation du réseau cible** est réalisée toutes les **500 étapes**, ce qui représente un compromis efficace entre stabilité et adaptabilité. Le **facteur d'escompte** est fixé à **0,95**, ce qui limite l'horizon temporel de décision de l'agent. Cette valeur permet de privilégier un raisonnement tactique à moyen terme (environ vingt coups), particulièrement adapté aux stratégies de capture et de promotion dans le jeu de Dama.

Troisièmement, l'**ingénierie des récompenses** joue un rôle déterminant dans le comportement de l'agent. La victoire finale reçoit la récompense maximale (**+1.0**). Cependant, pour faciliter l'apprentissage, des récompenses intermédiaires ont été introduites : **+0.3** pour une capture et **+0.5** pour la promotion d'un pion en dame, action stratégique majeure. Une pénalité légère de **−0.002** est appliquée à chaque mouvement afin d'éviter les comportements passifs et encourager l'agent à rechercher une victoire rapide.

La stratégie d'**exploration–exploitation** repose sur une politique **ε-greedy** avec décroissance exponentielle. Le taux d'exploration passe progressivement de **0.4** à **0.05** sur **70 % de l'entraînement** (soit environ l'épisode 1050 sur 1500). Le facteur de décroissance est calculé analytiquement à partir de la relation :

$$\varepsilon_{min} = \varepsilon_{start} \times decay^{1050}$$

ce qui donne un taux de décroissance d'environ **0.998022**. Le maintien d'un minimum de **5 % d'actions aléatoires** permet d'éviter que l'agent ne converge vers des stratégies sous-optimales.

Enfin, la **mémoire d'expérience (Replay Buffer)** est limitée à **100 000 transitions**. Une transition comprend deux états 8×88 \times 88×8 en nombres flottants, un masque d'actions de 4096 possibilités, une action, une récompense et un indicateur terminal. La taille estimée d'une transition est d'environ **16,9 Ko**, ce qui conduit à une utilisation mémoire maximale proche de **1,69 Go**. Cette valeur reste compatible avec les ressources disponibles dans l'environnement de calcul cloud utilisé.

Pour assurer la **reproductibilité expérimentale**, l'ensemble des hyperparamètres et des calculs théoriques est centralisé dans le code suivant :

```
import math
import numpy as np
import tensorflow as tf

# Hyperparamètres principaux
GAMMA = 0.95
BATCH_SIZE = 512
TARGET_UPDATE = 500
REPLAY_CAPACITY = 100000

# Récompenses
R_WIN = 1.0
R_CAPTURE = 0.3
R_PROMOTE = 0.5
R_STEP = -0.002

# Exploration epsilon-greedy
EPISODES_PHASE_B = 1500
EPSILON_START = 0.4
EPSILON_MIN = 0.05
EPSILON_DECAY_B = math.exp(math.log(EPSILON_MIN / EPSILON_START) /
(EPISODES_PHASE_B * 0.7))

# Planification du taux d'apprentissage (warm-up)
class WarmupSchedule(tf.keras.optimizers.schedules.LearningRateSchedule):
    def __init__(self, target_lr=2e-4, warmup_steps=2800):
        super(WarmupSchedule, self).__init__()
        self.target_lr = tf.cast(target_lr, tf.float32)
        self.warmup_steps = tf.cast(warmup_steps, tf.float32)

    def __call__(self, step):
        return self.target_lr * tf.minimum(1.0, tf.cast(step, tf.float32) /
self.warmup_steps)

# Optimiseur avec limitation du gradient
optimizer = tf.keras.optimizers.Adam(
    learning_rate=WarmupSchedule(target_lr=2e-4, warmup_steps=2800),
    clipnorm=1.0
)
```

Résumé :
Ce paramétrage vise à assurer un apprentissage stable et efficace du modèle Double DQN appliqué au jeu de Dama. Les choix effectués reposent sur trois principes principaux : stabilisation de l'apprentissage (batch large, warm-up, réseau cible), guidage comportemental (récompenses intermédiaires), et équilibre exploration–exploitation (décroissance contrôlée d'ε). Ensemble, ces éléments permettent d'améliorer la convergence et la robustesse de l'agent d'apprentissage.

Annexe 2 : algorithme Mini-MAX

L'algorithme MiniMax est une méthode de prise de décision utilisée en intelligence artificielle pour les jeux à deux joueurs comme le Tic-Tac-Toe se dessous. Il repose sur une exploration complète des coups possibles sous forme d'un arbre de jeu, où chaque nœud représente un état du plateau. Le joueur MAX, généralement l'intelligence artificielle, cherche à maximiser le score, tandis que le joueur MIN, l'adversaire, cherche à le minimiser. L'algorithme suppose que les deux joueurs jouent de manière optimale. À partir de l'état initial, il génère récursivement tous les états futurs possibles jusqu'aux positions finales du jeu. Chaque état terminal est ensuite évalué à l'aide d'une fonction de score qui attribue une valeur positive en cas de victoire de MAX, négative en cas de défaite, et nulle en cas d'égalité. Ces valeurs sont ensuite propagées de bas en haut dans l'arbre : les nœuds correspondant à MAX sélectionnent la valeur maximale parmi leurs enfants, tandis que ceux de MIN choisissent la valeur minimale. Ce processus permet de déterminer le meilleur coup à jouer depuis l'état initial, en garantissant le résultat optimal contre un adversaire parfait.

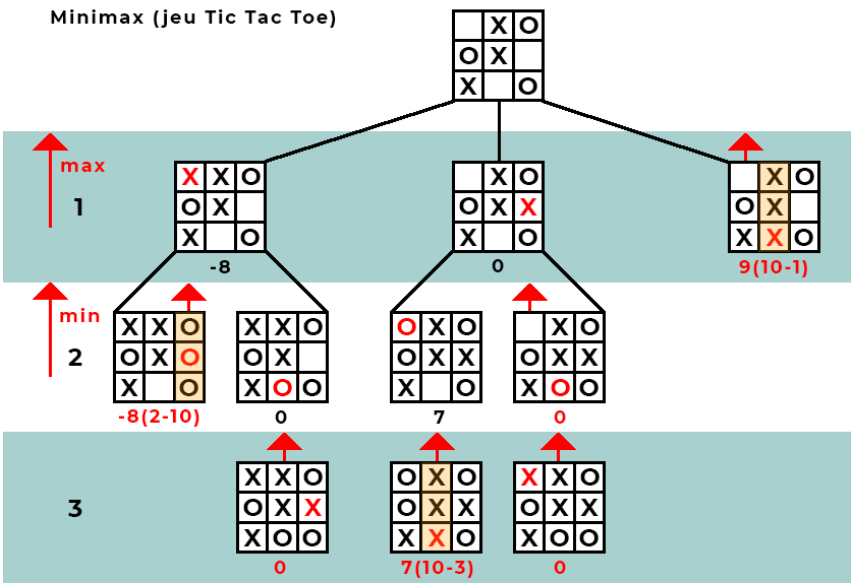


Figure 4 MiniMax Dans le cas du jeu (Tic-Tac-Toe)

Dans le cas du jeu XO (Tic-Tac-Toe), MiniMax examine toutes les configurations possibles du plateau après chaque coup. Par exemple, à partir d'une position donnée, il simule tous les coups possibles de X, puis pour chacun de ces coups, il simule les réponses possibles de O, et ainsi de suite jusqu'à atteindre des états finaux où la partie est gagnée, perdue ou nulle. Si une branche mène à une victoire rapide pour X, elle reçoit une valeur élevée, alors qu'une défaite ou une victoire tardive reçoit une valeur plus faible. Lors de la remontée des valeurs, si c'est le tour de X, l'algorithme choisit le coup qui maximise ses chances de gagner, tandis que si c'est le tour de O, il considère que l'adversaire choisira le coup le plus défavorable pour X. Ainsi, MiniMax permet à X de sélectionner le mouvement optimal en anticipant toutes les réactions possibles de O, ce qui conduit à une stratégie parfaite où X ne peut jamais perdre si l'algorithme est appliqué correctement.